# CC - Employee Management System using Apex in Salesforce

## Problem Statement

Develop a menu-driven Employee Management System in Salesforce using console-based Apex programming to manage employee records (Emp ID, Emp Name, Email, Birth Date, Department).

---

# 1. Login to Salesforce

Open:

https://login.salesforce.com

Login with your Salesforce Developer Account.

---

# 2. Open Setup

1. Click Gear Icon (⚙)
2. Click:

```
Setup
```

---

# 3. Create Custom Object

Go to:

```
Setup → Object Manager → Create → Custom Object
```

## Fill Object Details

| Field | Value |
| --- | --- |
| Label | Employee Record |

| Field | Value |
|---|---|
| Plural Label | Employee Records |
| Object Name | Employee_Record |
| Record Name | Employee Name |

Select:

- Allow Reports
- Allow Activities

Click:

```
Save
```

---

# 4. Create Custom Fields

Open:

```
Object Manager → Employee Record → Fields & Relationships
```

Now create the following fields one by one.

---

## A. Create Emp ID Field

1. Click:

```
New
```

1. Select:

```
Number
```

1. Click:

```
Next
```

1. Fill Details:

| Property | Value |
| --- | --- |
| Field Label | Emp ID |
| Length | 5 |
| Decimal Places | 0 |

1. Click:

```
Next → Save
```

API Name automatically becomes:

```
Emp_ID__c
```

---

## B. Create Email Field

1. Click:

```
New
```

1. Select:

```
Email
```

1. Click:

```
Next
```

1. Fill Details:

| Property | Value |
| --- | --- |
| Field Label | Email |

1. Click:

```
Next → Save
```

API Name:

```
Email__c
```

---

## C. Create Birth Date Field

1. Click:

```
New
```

1. Select:

```
Date
```

1. Click:

```
Next
```

1. Fill Details:

| Property | Value |
| --- | --- |
| Field Label | Birth Date |

1. Click:

```
Next → Save
```

API Name:

```
Birth_Date__c
```

---

## D. Create Department Field

1. Click:

```
New
```

1. Select:

```
Text
```

1. Click:

```
Next
```

1. Fill Details:

| Property | Value |
|---|---|
| Field Label | Department |
| Length | 30 |

1. Click:

```
Next → Save
```

API Name:

```
Department__c
```

---

## 5. Open Developer Console

Click:

```
Gear Icon → Developer Console
```

Developer Console opens in new window.

---

## 6. Create Apex Class

Go to:

```
File → New → Apex Class
```

Class Name:

```
EmployeeRecordManager
```

Click:

```
OK
```

---

# 7. Paste Apex Code

Delete default code and paste:

```
public class EmployeeRecordManager {

    // ADD EMPLOYEE
    public static void addEmployee(Integer empId, String empName, String email,
Date birthDate, String department) {

        Employee_Record__c e = new Employee_Record__c();

        e.Name = empName;
        e.Emp_ID__c = empId;
        e.Email__c = email;
        e.Birth_Date__c = birthDate;
        e.Department__c = department;

        insert e;
    }

    // VIEW EMPLOYEES
    public static void viewEmployees() {

        List<Employee_Record__c> employees = [SELECT Name, Emp_ID__c, Email__c,
Birth_Date__c, Department__c
                                              FROM Employee_Record__c];

        for(Employee_Record__c e : employees) {

            System.debug('Emp Name: ' + e.Name + ', Emp ID: ' + e.Emp_ID__c + ',
Email: ' + e.Email__c + ', Birth Date: ' + e.Birth_Date__c + ', Department: ' +
e.Department__c);
        }
    }
```

```
    // UPDATE EMPLOYEE
    public static void updateEmployee(Integer empId, String newName, String
newEmail, String newDepartment) {

        Employee_Record__c e = [SELECT Name, Emp_ID__c, Email__c, Department__c
                                FROM Employee_Record__c
                                WHERE Emp_ID__c = :empId LIMIT 1];

        e.Name = newName;
        e.Email__c = newEmail;
        e.Department__c = newDepartment;

        update e;
    }

    // DELETE EMPLOYEE
    public static void deleteEmployee(Integer empId) {

        Employee_Record__c e = [SELECT Id
                                FROM Employee_Record__c
                                WHERE Emp_ID__c = :empId LIMIT 1];

        delete e;
    }
}
```

---

# 8. Save Apex Class

Press:

```
Ctrl + S
```

---

# 9. Open Execute Anonymous Window

Go to:

```
Debug → Open Execute Anonymous Window
```

Shortcut:

```
Ctrl + E
```

---

# 10. Add Employee Record

```
EmployeeRecordManager.addEmployee(
    101,
    'Abhishek',
    'abhi@gmail.com',
    Date.newInstance(2004, 5, 10),
    'IT'
);
```

Click Execute.

---

# 11. View Employee Records

```
EmployeeRecordManager.viewEmployees();
```

Execute.

---

# 12. Check Output

Open:

```
Logs → Debug Only
```

Example Output:

```
Emp Name: Abhishek, Emp ID: 101, Email: abhi@gmail.com, Birth Date: 2004-05-10,
Department: IT
```

---

## 13. Update Employee Record

```
EmployeeRecordManager.updateEmployee(
    101,
    'Abhishek Bobade',
    'abhishek@gmail.com',
    'HR'
);
```

Execute.

---

## 14. Delete Employee Record

```
EmployeeRecordManager.deleteEmployee(101);
```

Execute.

---

## 15. Conclusion

The Employee Management System was successfully implemented using Apex in Salesforce. The application performs CRUD operations on employee records containing Emp ID, Employee Name, Email, Birth Date, and Department using console-based execution in Salesforce.